# EMACS, THE ULTIMATE

*Ta Quang Trung*

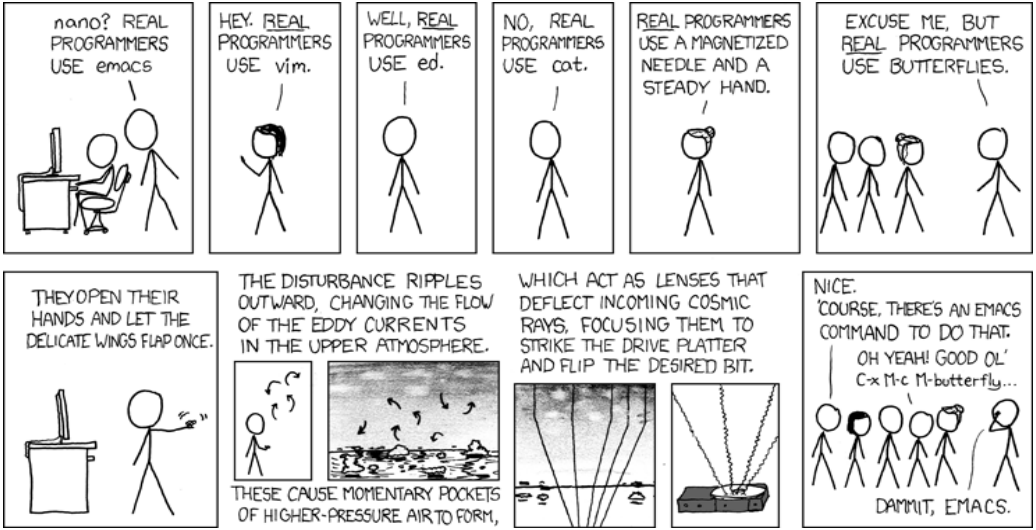
*National University of Singapore*

*Singapore, October 27th, 2017*

Which editor do you use the most?

# CASE STUDY: REAL PROGRAMMERS

# CASE STUDY: REAL PROGRAMMERS




https://xkcd.com/378/

# M-x butterfly

# Outline


1. Emacs, the basic
2. Emacs, the advance
3. Emacs, the ultimate
4. Getting started
5. Conclusion

# A few notes

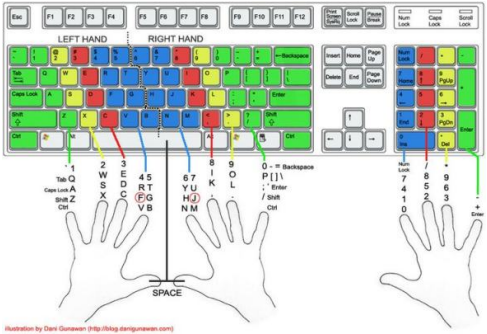
- The talk title is borrowed from: `lambda-the-ultimate.org`
- Introduce what I know that Emacs can do
- No intention to compare Emacs with other editors/IDEs
- Feel free to interrupt me for questions, demo requests, etc

# Emacs, the basic

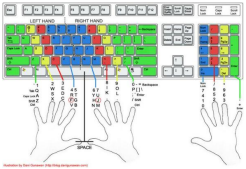
# How do we interact with editor?

# How do we interact with editor?

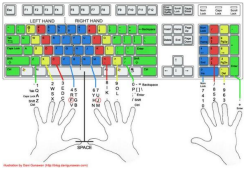
# How do we interact with editor?



# How can we edit text unproductively?

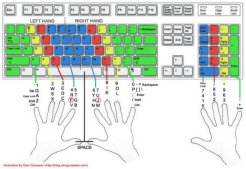


# How can we edit text unproductively?



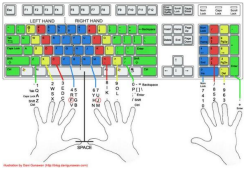

- Frequently moving hand between keyboard & mouse

# How can we edit text unproductively?



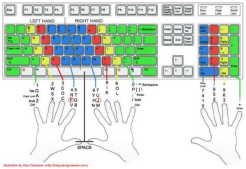

- Frequently moving hand between keyboard & mouse
- Frequently moving palm around keyboard keys

# How can we edit text unproductively?




- Frequently moving hand between keyboard & mouse
- Frequently moving palm around keyboard keys
- Not using convenient editing features
  - Use $\leftarrow, \rightarrow, \uparrow, \downarrow$ keys instead of Home, End, PageUp, PageDown
  - Use an editor with few convenient editing features

# How can we edit text unproductively?




- Frequently moving hand between keyboard & mouse
- Frequently moving palm around keyboard keys
- Not using convenient editing features
  - Use $\leftarrow, \rightarrow, \uparrow, \downarrow$ keys instead of Home, End, PageUp, PageDown
  - Use an editor with few convenient editing features

$\Rightarrow$ Emacs helps to avoid these problems!

# What is Emacs?

- A text editor born in 1976 and still lives actively today
- The name Emacs stands for Editor MACroS
- The first Emacs version was written in 1976 by David A. Moon and Guy L. Steele Jr
- The most popular version GNU Emacs was created by Richard Stallman in 1984
- Many modern editors such as Eclipse, IntelliJ IDEA, Sublime, Visual Studio have an Emacs simulation mode

# Emacs movements

# Emacs movements

- Use $\leftarrow, \rightarrow, \uparrow, \downarrow$, Home, End, PageUp, PageDown as usual

# Emacs movements

- Use $\leftarrow, \rightarrow, \uparrow, \downarrow$, Home, End, PageUp, PageDown as usual
- But b, f, p, n, a, e, v can avoid palm moving
  (**b**ackward/**f**orward/**p**revious/**n**ext /**a** ... z/**e**nd/**v** in $\downarrow$)

# Emacs movements

- Use $\leftarrow, \rightarrow, \uparrow, \downarrow$, Home, End, PageUp, PageDown as usual
- But b, f, p, n, a, e, v can avoid palm moving (**b**ackward/**f**orward/**p**revious/**n**ext /**a** ... z/**e**nd/**v** in $\downarrow$)
  - `C-b`, `C-f`: backward/forward character (`C` denotes the Ctrl key)
  - `C-p`, `C-n`, `C-a`, `C-e`: previous/next line, begin/end of line
  - `C-v`, `M-v`: scroll up/down window (M denotes the Alt key)
  - `M-b`, `M-f`: backward/forward word

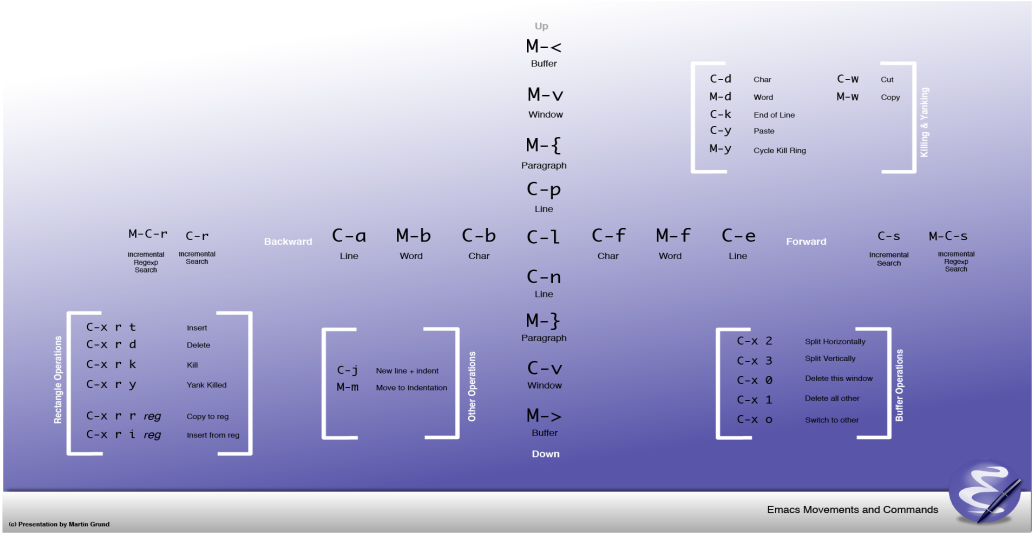
# Emacs movements

- Use $\leftarrow, \rightarrow, \uparrow, \downarrow$, Home, End, PageUp, PageDown as usual
- But b, f, p, n, a, e, v can avoid palm moving (**b**ackward/**f**orward/**p**revious/**n**ext /**a** ... z/**e**nd/**v** in $\downarrow$)
  - `C-b`, `C-f`: backward/forward character (`C` denotes the Ctrl key)
  - `C-p`, `C-n`, `C-a`, `C-e`: previous/next line, begin/end of line
  - `C-v`, `M-v`: scroll up/down window (M denotes the Alt key)
  - `M-b`, `M-f`: backward/forward word
- Too complicated? Use the cheatsheet!

# Emacs movements

- Use $\leftarrow, \rightarrow, \uparrow, \downarrow$, Home, End, PageUp, PageDown as usual
- But b, f, p, n, a, e, v can avoid palm moving (**b**ackward/**f**orward/**p**revious/**n**ext /**a** ... z/**e**nd/**v** in $\downarrow$)
  - `C-b`, `C-f`: backward/forward character (`C` denotes the Ctrl key)
  - `C-p`, `C-n`, `C-a`, `C-e`: previous/next line, begin/end of line
  - `C-v`, `M-v`: scroll up/down window (M denotes the Alt key)
  - `M-b`, `M-f`: backward/forward word
- Too complicated? Use the cheatsheet!
- Additional benefit: Unix/Linux Shell uses the same movement keys like Emacs!

# Emacs movements




By Martin Grand

# Buffer and window

# Buffer and window

- Emacs buffer:
  - The Emacs object containing text
  - Buffer $\neq$ file: a file can be opened in multiple buffers
  - `C-x C-f`, `C-x C-b`, `C-x k`: open file, switch buffer, kill buffer

# Buffer and window

- Emacs buffer:
  - The Emacs object containing text
  - Buffer $\neq$ file: a file can be opened in multiple buffers
  - `C-x C-f`, `C-x C-b`, `C-x k`: open file, switch buffer, kill buffer
- Emacs window:
  - The Emacs object showing a buffer
  - Emacs' window $\neq$ window in Linux/Windows (Emacs calls it *frame*)
  - `C-x 0, 1, 2, 3`: delete, maximize, split horizontally/vertically
  - `C-x {, }, ^, _`: shrink, enlarge horizontally/vertically

# Mark, region and kill ring

# Mark, region and kill ring

- Mark and region
  - Mark is a special text position set by the command `C-Space`
  - Region is the text between the point (current cursor) and the mark
  - Flexibility: set mark once and adjust the region by moving the cursor

# Mark, region and kill ring

- Mark and region
  - Mark is a special text position set by the command `C-Space`
  - Region is the text between the point (current cursor) and the mark
  - Flexibility: set mark once and adjust the region by moving the cursor
- Kill ring
  - The greedy clipboard of Emacs
  - `M-w`, `C-w`, `C-y`: copy, cut, yank (paste)
  - `M-y`: paste the previously copied/cut texts
  - `C-k`, `M-d`, `C-Backspace`, `C-Del`, ... store deleted texts into kill ring

# Emacs, the advance

# Advanced editing

# Advanced editing

- Edit, mark by symbolic expressions: `smartparens`

# Advanced editing

- Edit, mark by symbolic expressions: `smartparens`
- Text completion: `hippie-expand`, `dabbrev-expand`

# Advanced editing

- Edit, mark by symbolic expressions: `smartparens`
- Text completion: `hippie-expand`, `dabbrev-expand`
- Using macro: `F3, F4`

# Advanced editing

- Edit, mark by symbolic expressions: `smartparens`
- Text completion: `hippie-expand`, `dabbrev-expand`
- Using macro: `F3, F4`
- Block editing: `C-x SPC`

# Advanced editing

- Edit, mark by symbolic expressions: `smartparens`
- Text completion: `hippie-expand`, `dabbrev-expand`
- Using macro: `F3, F4`
- Block editing: `C-x SPC`
- Text undo:
  - Default undo: greedy, remember everything: undo of undo of undo ...
  - Modern undo: `undo-tree` with visualization

# Advanced editing

# Advanced editing

- Text conversion and transposition
  - `M-l`, `M-u`, `M-c`: **l**owercase, **u**ppercase, **c**apitalize first character
  - `C-t`, `M-t`, `C-M-t`, `C-x C-t`: transpose character, word, expression, line

# Advanced editing

- Text conversion and transposition
  - `M-l`, `M-u`, `M-c`: **l**owercase, **u**ppercase, **c**apitalize first character
  - `C-t`, `M-t`, `C-M-t`, `C-x C-t`: transpose character, word, expression, line
- Text search, replace and highlighting in current buffer:
  - Search: `isearch`, `swivel`, `occur`
  - Replace: `anzu`
  - Highlight: `highlighting`
  - Regular expression supported

# Handy commands

# Handy commands

- Emacs commands completion and narrowing: `helm`, `ivy`

# Handy commands

- Emacs commands completion and narrowing: `helm`, `ivy`
- Key-binding suggestion: `which-key`

# Handy commands

- Emacs commands completion and narrowing: `helm`, `ivy`
- Key-binding suggestion: `which-key`
- Text navigation in a few key strokes: `avy`

# Handy commands

- Emacs commands completion and narrowing: `helm`, `ivy`
- Key-binding suggestion: `which-key`
- Text navigation in a few key strokes: `avy`
- Window selection: `other-window`, `windmove`, `winum`, `winner`

# Handy commands

- Emacs commands completion and narrowing: `helm`, `ivy`
- Key-binding suggestion: `which-key`
- Text navigation in a few key strokes: `avy`
- Window selection: `other-window`, `windmove`, `winum`, `winner`
- Buffer navigation: `helm-mini`, `previous/next buffer`

# Handy commands

- Emacs commands completion and narrowing: `helm`, `ivy`
- Key-binding suggestion: `which-key`
- Text navigation in a few key strokes: `avy`
- Window selection: `other-window`, `windmove`, `winum`, `winner`
- Buffer navigation: `helm-mini`, `previous/next buffer`
- Working with other-window: `<command>-other-window`

# Handy commands

- Emacs commands completion and narrowing: `helm`, `ivy`
- Key-binding suggestion: `which-key`
- Text navigation in a few key strokes: `avy`
- Window selection: `other-window`, `windmove`, `winum`, `winner`
- Buffer navigation: `helm-mini`, `previous/next buffer`
- Working with other-window: `<command>-other-window`
- Others: `just-one-space`, `delete-blank-lines`, `repeat`

# Understand Emacs' components

- Buffer, window, frame
- Major mode: display buffer, visualize and manipulate content
- Minor mode: auxiliary editing and visualizing features
- Mode line, header line: all auxiliary information
- Mini-buffer, echo area: user interacts with Emacs commands

# Explore Emacs' documentation

- Describe major mode: `C-h m` (`M-x describe-mode`)
- Describe key binding: `C-h k` (`M-x describe-key`)
- Describe function: `C-h f` (`M-x describe-function`)
- Describe variable: `C-h v` (`M-x describe-variable`)

# Emacs, the ultimate

# Integrated Development Environment

# Integrated Development Environment

- Support almost every programming languages:
  - E.g., C/C++, OCaml, Latex, Python, HTML, Java, LLVM bitcode
  - Comfortability: (almost) the same key bindings for all languages

# Integrated Development Environment

- Support almost every programming languages:
  - E.g., C/C++, OCaml, Latex, Python, HTML, Java, LLVM bitcode
  - Comfortability: (almost) the same key bindings for all languages
- Syntax highlighting, auto indentation

# Integrated Development Environment

- Support almost every programming languages:
  - E.g., C/C++, OCaml, Latex, Python, HTML, Java, LLVM bitcode
  - Comfortability: (almost) the same key bindings for all languages
- Syntax highlighting, auto indentation
- Code completion, code outline, jump to definition

# Integrated Development Environment

- Support almost every programming languages:
  - E.g., C/C++, OCaml, Latex, Python, HTML, Java, LLVM bitcode
  - Comfortability: (almost) the same key bindings for all languages
- Syntax highlighting, auto indentation
- Code completion, code outline, jump to definition
- Lint checking (on-the-fly), syntax checking (on-the-fly)

# Integrated Development Environment

- Support almost every programming languages:
  - E.g., C/C++, OCaml, Latex, Python, HTML, Java, LLVM bitcode
  - Comfortability: (almost) the same key bindings for all languages
- Syntax highlighting, auto indentation
- Code completion, code outline, jump to definition
- Lint checking (on-the-fly), syntax checking (on-the-fly)
- Compilation mode: jump back and forth to errors

# Integrated Development Environment

- Support almost every programming languages:
  - E.g., C/C++, OCaml, Latex, Python, HTML, Java, LLVM bitcode
  - Comfortability: (almost) the same key bindings for all languages
- Syntax highlighting, auto indentation
- Code completion, code outline, jump to definition
- Lint checking (on-the-fly), syntax checking (on-the-fly)
- Compilation mode: jump back and forth to errors
- *Weakness*: code refactoring is limited

# Integrated Development Environment

- Project management: `speedbar`, `projectile`
  - Navigate among files and folders
  - Search files in a project, search/replace text in files of a project
  - Switch between parent and child projects

# Integrated Development Environment

- Project management: `speedbar`, `projectile`
  - Navigate among files and folders
  - Search files in a project, search/replace text in files of a project
  - Switch between parent and child projects
- Version control: Git, Hg, CVS, Svn, Bazaar, Monotone, ...
  - Git: `magit` – everything that git cli can do
  - Hg: `monky` – influenced by magit

# Integrated Development Environment

- Project management: `speedbar`, `projectile`
  - Navigate among files and folders
  - Search files in a project, search/replace text in files of a project
  - Switch between parent and child projects
- Version control: Git, Hg, CVS, Svn, Bazaar, Monotone, ...
  - Git: `magit` – everything that git cli can do
  - Hg: `monky` – influenced by magit
- Terminal: `shell`, `term`, `eshell`

# Integrated Development Environment

- Project management: `speedbar`, `projectile`
  - Navigate among files and folders
  - Search files in a project, search/replace text in files of a project
  - Switch between parent and child projects
- Version control: Git, Hg, CVS, Svn, Bazaar, Monotone, ...
  - Git: `magit` – everything that git cli can do
  - Hg: `monky` – influenced by magit
- Terminal: `shell`, `term`, `eshell`
- Text diff and merge: `diff`, `ediff`

# Everyday Emacs

# Everyday Emacs

- File manager:
  - Directory as a buffer: `dired`
  - Recent files, recent directories: `recentf`

# Everyday Emacs

- File manager:
  - Directory as a buffer: `dired`
  - Recent files, recent directories: `recentf`
- Search, replace and highlight anywhere in the file system
  - Text search & replace: `grep`, `helm-grep`, `helm-ag`, `swiper`, `anzu`
  - Text highlighting: `isearch`, `highlight-regex`

# Everyday Emacs

- File manager:
  - Directory as a buffer: `dired`
  - Recent files, recent directories: `recentf`
- Search, replace and highlight anywhere in the file system
  - Text search & replace: `grep`, `helm-grep`, `helm-ag`, `swiper`, `anzu`
  - Text highlighting: `isearch`, `highlight-regex`
  - File search: `helm-locate`, `projectile-find-file`, `helm-recentf`

# Everyday Emacs

- Workspace management: `eyebrowse`

# Everyday Emacs

- Workspace management: `eyebrowse`
- Note taking, project planning: `org-mode`

# Everyday Emacs

- Workspace management: `eyebrowse`
- Note taking, project planning: `org-mode`
- Document viewing: images, pdf, presentation, archive files

# Everyday Emacs

- Workspace management: `eyebrowse`
- Note taking, project planning: `org-mode`
- Document viewing: images, pdf, presentation, archive files
- Spell checking: `aspell`, `hunspell`, `flyspell`, `langtool`

# Everyday Emacs

- Workspace management: `eyebrowse`
- Note taking, project planning: `org-mode`
- Document viewing: images, pdf, presentation, archive files
- Spell checking: `aspell`, `hunspell`, `flyspell`, `langtool`
- Web browser, remote access, email reader: `eww`, `tramp`, `gnus`

# Everyday Emacs

- Workspace management: `eyebrowse`
- Note taking, project planning: `org-mode`
- Document viewing: images, pdf, presentation, archive files
- Spell checking: `aspell`, `hunspell`, `flyspell`, `langtool`
- Web browser, remote access, email reader: `eww`, `tramp`, `gnus`
- More handy, useful features, and even game, screensaver!

# Getting started

# Emacs Starter Kits

| Name | Stars | Forks | Contri-butors | Commits | GitHub Sites |
|---|---|---|---|---|---|
| **Spacemacs** | 12.430 | 3.219 | 574 | 7.405 | `GH/syl20bnr/spacemacs` |
| **Purcell emacs** | 3.668 | 1.866 | 16 | 2.638 | `GH/purcell/emacs.d` |
| **Prelude** | 3.470 | 1.453 | 120 | 1.239 | `GH/bbatsov/prelude` |
| **Emacs live** | 1.240 | 245 | 34 | 703 | `GH/overtone/emacs-live` |
| **Magnars emacs** | 1.135 | 206 | 8 | 1.402 | `GH/magnars/.emacs.d` |

Updated in August 05$^{th}$, 2017

# Create your own style Emacs

# Create your own style Emacs

- Simple customization: GUI supported
  - M-x customize-mode, M-x customize-group, etc

# Create your own style Emacs

- Simple customization: GUI supported
  - M-x customize-mode, M-x customize-group, etc
- Useful Emacs packages and libraries:
  - `https://github.com/emacs-tw/awesome-emacs`

# Create your own style Emacs

- Simple customization: GUI supported
  - M-x customize-mode, M-x customize-group, etc
- Useful Emacs packages and libraries:
  - `https://github.com/emacs-tw/awesome-emacs`
- Write your own Emacs functions or packages:
  - Need to be familiar with Emacs Lisp language
  - Starting point: copy an existing function and modify it

# Be aware

# Be aware

- Emacs pinky! Which key is used the most? How do you type it?

# Be aware

- Emacs pinky! Which key is used the most? How do you type it?
  - Solution: use god-mode, hydra-mode, or a foot pedal!

# Be aware

- Emacs pinky! Which key is used the most? How do you type it?
  - Solution: use god-mode, hydra-mode, or a foot pedal!
- Emacs bankruptcy: `.emacs` will be out of control one day

# Be aware

- Emacs pinky! Which key is used the most? How do you type it?
  - Solution: use god-mode, hydra-mode, or a foot pedal!
- Emacs bankruptcy: `.emacs` will be out of control one day
  - Solution: use Emacs starter kits

# Be aware

- Emacs pinky! Which key is used the most? How do you type it?
  - Solution: use god-mode, hydra-mode, or a foot pedal!
- Emacs bankruptcy: `.emacs` will be out of control one day
  - Solution: use Emacs starter kits
- Ctrl-Alt-Backspace: don't try it on your Linux

# Be aware

- Emacs pinky! Which key is used the most? How do you type it?
  - Solution: use god-mode, hydra-mode, or a foot pedal!
- Emacs bankruptcy: `.emacs` will be out of control one day
  - Solution: use Emacs starter kits
- Ctrl-Alt-Backspace: don't try it on your Linux
  - Solution: `sudo dpkg-reconfigure keyboard-configuration`

# Be aware

- Emacs pinky! Which key is used the most? How do you type it?
  - Solution: use god-mode, hydra-mode, or a foot pedal!
- Emacs bankruptcy: `.emacs` will be out of control one day
  - Solution: use Emacs starter kits
- Ctrl-Alt-Backspace: don't try it on your Linux
  - Solution: `sudo dpkg-reconfigure keyboard-configuration`
- Malicious/bad code from third-party packages

# Be aware

- Emacs pinky! Which key is used the most? How do you type it?
  - Solution: use god-mode, hydra-mode, or a foot pedal!
- Emacs bankruptcy: `.emacs` will be out of control one day
  - Solution: use Emacs starter kits
- Ctrl-Alt-Backspace: don't try it on your Linux
  - Solution: `sudo dpkg-reconfigure keyboard-configuration`
- Malicious/bad code from third-party packages
  - Solution: do not use if they looks suspicious

# Be aware

- Emacs pinky! Which key is used the most? How do you type it?
  - Solution: use god-mode, hydra-mode, or a foot pedal!
- Emacs bankruptcy: `.emacs` will be out of control one day
  - Solution: use Emacs starter kits
- Ctrl-Alt-Backspace: don't try it on your Linux
  - Solution: `sudo dpkg-reconfigure keyboard-configuration`
- Malicious/bad code from third-party packages
  - Solution: do not use if they looks suspicious
- Emacs customization may be addicted

# Be aware

- Emacs pinky! Which key is used the most? How do you type it?
  - Solution: use god-mode, hydra-mode, or a foot pedal!
- Emacs bankruptcy: `.emacs` will be out of control one day
  - Solution: use Emacs starter kits
- Ctrl-Alt-Backspace: don't try it on your Linux
  - Solution: `sudo dpkg-reconfigure keyboard-configuration`
- Malicious/bad code from third-party packages
  - Solution: do not use if they looks suspicious
- Emacs customization may be addicted
  - Solution: positive thinking – it could be fun!

# Conclusion

# Conclusion

- Introduce the versatility of Emacs and how to get started

# Conclusion

- Introduce the versatility of Emacs and how to get started
- Advantages:

# Conclusion

- Introduce the versatility of Emacs and how to get started
- Advantages:
  - Editor + file manager, project manager, terminal, multimedia, etc

# Conclusion

- Introduce the versatility of Emacs and how to get started
- Advantages:
  - Editor + file manager, project manager, terminal, multimedia, etc
  - Big and active community

# Conclusion

- Introduce the versatility of Emacs and how to get started
- Advantages:
  - Editor + file manager, project manager, terminal, multimedia, etc
  - Big and active community
- Should you use Emacs?

# Conclusion

- Introduce the versatility of Emacs and how to get started
- Advantages:
  - Editor + file manager, project manager, terminal, multimedia, etc
  - Big and active community
- Should you use Emacs?
  - Depends on personal taste but Emacs is worth to be given a try!

# Conclusion

- Introduce the versatility of Emacs and how to get started
- Advantages:
  - Editor + file manager, project manager, terminal, multimedia, etc
  - Big and active community
- Should you use Emacs?
  - Depends on personal taste but Emacs is worth to be given a try!

Thank you!

# Conclusion

- Introduce the versatility of Emacs and how to get started
- Advantages:
  - Editor + file manager, project manager, terminal, multimedia, etc
  - Big and active community
- Should you use Emacs?
  - Depends on personal taste but Emacs is worth to be given a try!

Thank you!

Questions?